

자료구조 및 알고리즘

Chapter 3. 알고리즘의 성능

3.1 알고리즘 수행 시간이란

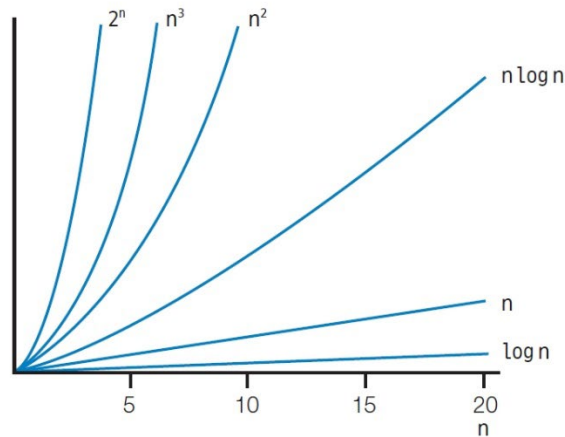
3.2 알고리즘 복잡도

1. 알고리즘 수행 시간이란

■ 알고리즘 수행 시간

- 입력의 크기가 n 에 대해 알고리즘을 수행하는데 걸리는 시간으로 표현
- 입력의 크기
 - 정렬: 정렬할 원소의 수
 - 색인: 색인에 포함된 원소의 수

→ 자원을 얼마나 효율적으로 사용하는 지에 대한 판단을 할 수 있음



n	$0.1n^2$	$0.1n^2+n+100$
10	10	120
20	40	160
50	250	400
100	1,000	1,200
1,000	100,000	101,100

최고차항이 2차인 경우 n 이 커지면
궁극적으로 2차 항이 지배한다.

그림 3-1 여러 함수의 증가율과 점근적 개념

■ 점심 뭐 먹을 지 정하기

- 1. 즉시 정하기
- 2. 단계적으로 줄여 나가기
- 3. 하나씩 메뉴 확인하면서 고르기
- 4. 모든 조합을 고려해서 최적의 코스 정하기

福 메뉴판		
☞ 면 류 ☜		
짜장면 8,000	칼국수 7,000	우동 7,500
간짜장 7,000	짜장면 10,000	삼선우동 12,000
양반짜장 9,000	삼선칼국수 12,000	삼선우면 8,000
유니짜장 11,000	삼선백종면 12,000	삼선소면 12,000
삼선짜장 11,000	고추칼국수 12,000	기스면 10,000
사면짜장 11,000	부나칼국수 9,000	떡면 11,000
불마는짜장 10,000	생굴칼국수 12,000	누룽지탕(식사) 12,000
유슬짜장 11,000	동갈칼국수(계란) 10,000	자른칼국수 12,000
삼선낙지칼국수 15,000	칼국수 7,000	배달(배)면 8,000
탕칼(배)면 10,000	탕배 10,000	당국양면(계란) 12,000
탕국수(계란) 8,000	냉면(계란) 8,000	군만두 5,000
물만두 5,000		
☞ 밥 류 ☜		
짜장밥 7,000	칼국수밥 7,500	짜장밥 7,000
계산짜장밥 9,000	삼선칼국수밥 12,000	우동밥 7,500
마파두부밥 9,000	칼국수 9,000	송이돌밥 11,000
삼선짜장밥 12,000	세우짜장밥 12,000	고추잡채밥 12,000
떡밥 12,000	유산술밥 15,000	칼국수 15,000
굴칼국수(계란) 12,000	속개장 8,000	갈비탕 8,000
☞ 요리 부 ☜		
탕수육 22,000	사면탕수육 24,000	매운탕수육 24,000
갈비재 22,000	라조육 27,000	간통육 27,000
간소육 27,000	라조기 27,000	간통기 27,000
난자참스 27,000	양장피 27,000	고추잡채 27,000
삼선누룽지탕 35,000	간소세우 35,000	간통세우 35,000
밀보재 35,000	갈비탕 35,000	유산술 35,000
삼선술국 17,000	* 갈비기, 궁기밥 추가 : 2,000	

```
sample1(A[], n):  
  k ← n/2  
  return A[k]
```

상수 시간
 $O(1)$

```
sample2(A[], n):  
  sum ← 0  
  for i ← 0 to n-1  
    sum ← sum + A[i]  
  return sum
```

n 에 비례하는 시간

$O(n)$

```
sample3(A[], n):  
  sum ← 0  
  for i ← 0 to n-1  
    for j ← 0 to n-1  
      sum ← sum + A[i] * A[j]  
  return sum
```

n^2 에 비례하는 시간

$O(n^2)$

sample4(A[], n):

sum $\leftarrow$ 0

for i $\leftarrow$ 0 **to** n-1

for j $\leftarrow$ 0 **to** n-1

k $\leftarrow$ A[0...n-1]에서 임의로 n/2개를 뽑은 것들

중 최댓값

sum $\leftarrow$ sum + k

return sum

$n \cdot n \cdot n$
for for (n개 점속)

— n^3 에 비례하는 시간

sample5(A[], n):

sum $\leftarrow$ 0

for i $\leftarrow$ 0 **to** n-2

for j $\leftarrow$ i+1 **to** n-1

sum $\leftarrow$ sum + A[i] * A[j]

return sum

$n-2$ n
1 2
0 1

— n^2 에 비례하는 시간

factorial(n):

if (n = 1) **return** 1

return n * **factorial**(n- 1)

— n에 비례하는 시간

2. 알고리즘 복잡도

■ 점근적 복잡도 Asymptotic Complexity

- 입력의 크기가 충분히 클 때의 복잡도

(n이 커지면)

- 알고리즘이 **가끔해야** n^2 에 비례하는 시간이 든다 $\rightarrow O(n^2)$

(상한)

- 알고리즘이 ~~가끔해야~~ n^2 에 비례하는 시간이 든다 $\rightarrow \Omega(n^2)$

비교

적어도, 못해도 (하한)

비교 개수

- 알고리즘이 **항상** n^2 에 비례하는 시간이 든다 $\rightarrow \Theta(n^2)$

비교 세타.

외지 전복 알고리즘 있어야지.

- **O-표기**

- $O(n^2)$: 최고차항의 차수가 n^2 을 넘지 않는 모든 함수의 집합

- **Ω -표기**

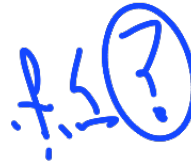
- $\Omega(n^2)$: 최고차항의 차수가 n^2 보다 작지 않은 모든 함수의 집합

- **Θ -표기**

- $\Theta(n^2)$: 최고차항의 차수가 n^2 인 모든 함수의 집합

$O(f(n))$: 빅 오 Big Oh

- 최고차항의 차수가 $f(n)$ 보다 작거나 같은 모든 함수
- 점근적 상한
- 예, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, ...
- $O(n^2) = \{n^2, 5n^2+7n+12, 100n^2-2n+9, n \log n+5n, 3n+9, 150, \dots\}$
 - 최고차항의 차수가 2차 이하인 모든 함수
- $5n^2+7n+12 \in O(n^2)$ 으로 표기해야 하나 편의상 $5n^2+7n+12 = O(n^2)$ 으로 표기함



$\Omega(f(n))$: 빅 오메가 Big Omega

- 최고차항의 차수가 $f(n)$ 보다 크거나 같은 모든 함수
- 점근적 하한
- 예, $\Omega(n)$, $\Omega(n \log n)$, $\Omega(n^2)$, $\Omega(2^n)$, ...
- $\Omega(n^2) = \{n^2, 5n^2+7n+12, 100n^2-2n+9, n^3+5n, 7n^5+n^3+9, 2^n, n!, \dots\}$
 - 최고차항의 차수가 2차 이상인 모든 함수
- $5n^3+12 \in \Omega(n^2)$ 으로 표기해야 하나 편의상 $5n^3+12 = \Omega(n^2)$ 으로 표기함

$\Theta(f(n))$: 빅 세타 Big Theta

- 최고차항의 차수가 $f(n)$ 과 동일한 모든 함수
- 예, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$, $\Theta(2^n)$, ...
- $\Theta(n^2) = \{n^2, 5n^2+7n+12, 100n^2-2n+9, \dots\}$
- 최고차항의 차수가 2차인 모든 함수
- $5n^2+12 \in \Theta(n^2)$ 으로 표기해야 하나 편의상 $5n^2+12 = \Theta(n^2)$ 으로 표기함

sample4(A[], n):

sum $\leftarrow$ 0

$n \rightarrow$ **for** $i \leftarrow 0$ **to** $n-1$

$n \rightarrow$ **for** $j \leftarrow 0$ **to** $n-1$

$n \rightarrow$ $k \leftarrow A[0 \dots n-1]$ 에서 임의로 $n/2$ 개를 뽑은 것들 중 최댓값
sum $\leftarrow$ sum + k

return sum

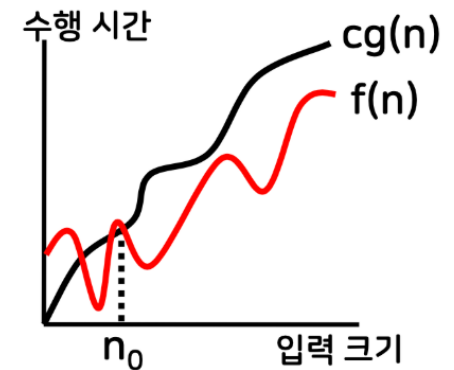
알고리즘 sample4()의 수행 시간은 $\Theta(n^3)$ 이다

- 알고리즘 점근적 복잡도 표현

$$f(n) = O(g(n))$$

모든 $n \geq n_0$ 에 대해,

$$f(n) \leq cg(n)$$



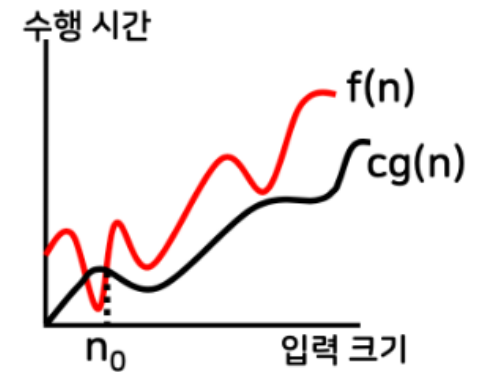
를 만족하는 어떤 양의 상수 c 와 n_0 이 존재한다.

- 알고리즘 점근적 복잡도 표현

$$f(n) = \Omega(g(n))$$

모든 $n \geq n_0$ 에 대해,

$$f(n) \geq cg(n)$$



를 만족하는 어떤 양의 상수 c 와 n_0 이 존재한다.

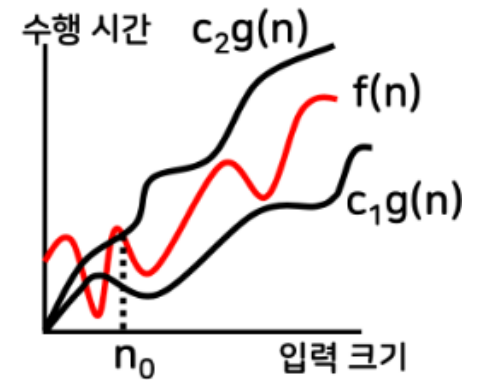
알게되면
풀어야한다.

- 알고리즘 점근적 복잡도 표현

$$f(n) = \Theta(g(n))$$

모든 $n \geq n_0$ 에 대해,

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$



를 만족하는 어떤 양의 상수 c_1 , c_2 와 n_0 이 존재한다.

- **점근적 표기에서 =는 ϵ 의 대용**

- 어떤 알고리즘의 수행 시간 $T(n) = O(n \log n)$ 이라 하면 $T(n) \in O(n \log n)$
- 즉, $T(n)$ 은 $O(n \log n)$ 에 속하는 함수란 뜻이다.
- 그러므로, $O(n \log n) = T(n)$ 과 같은 표현은 적절하지 않다.

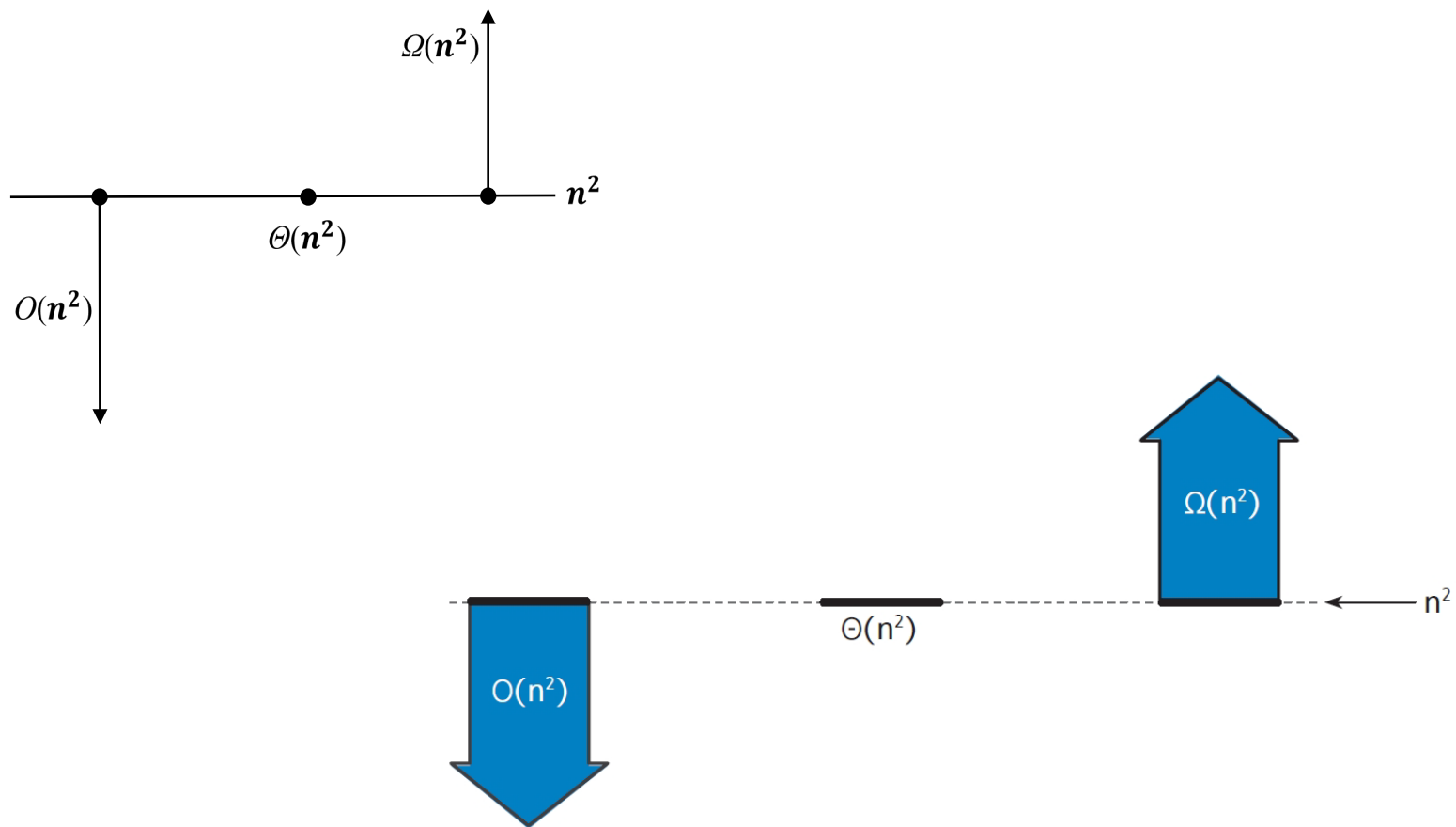


그림 3-3 점근적 표현의 시각적 설명

```
alg1(A[], n):  
  k ← n/2  
  return A[k]
```

n에 관계없이 상수 시간이 소요된다
 $\Omega(1)$ 이기도 하고 $O(1)$ 이기도 하다
 $\Theta(1)$: 가장 정확한 표현

```
alg2(A[], n):  
  sum ← 0  
  for i ← 0 to n-1  
    sum ← sum + A[i]  
  return sum
```

항상 n에 비례하는 시간이 소요된다
 $\Omega(n)$ 이기도 하고 $O(n)$ 이기도 하다
 $\Theta(n)$: 가장 정확한 표현

alg3(A[], n) :

sum $\leftarrow$ 0

if (n0 | 홀수)

for i $\leftarrow$ 0 **to** n-1

sum $\leftarrow$ sum + A[i]

return sum

최악의 경우
최선의 경우

최악의 경우 n에 비례하는 시간이 소요된다

최선의 경우 상수 시간이 소요된다

$\Omega(1)$ 이고 $O(n)$ 이다

최선의 경우 $\Theta(1)$, 최악의 경우 $\Theta(n)$ 이라고 말한다

alg4(A[], n):

sum $\leftarrow$ 0

for i $\leftarrow$ 0 **to** n-2 (n-1)

for j $\leftarrow$ i+1 **to** n-1 (n-1)

sum $\leftarrow$ sum + A[i] * A[j]

return sum

$(n-1)(n-1) + (n-1)(n-2) \dots (n-1)$

항상 n^2 에 비례하는 시간이 소요된다

— $\Omega(n^2)$ 이기도 하고 $O(n^2)$ 이기도 하다

$\Theta(n^2)$: 가장 정확한 표현

이진탐색

```
binarySearch (A[], x, low, high) :  
    // A: array, x: search key, low, high: array bounds  
    if (low > high) return "Not found"  
    mid ← (low + high)/2  
    if (A[mid] < x) return binarySearch(A, x, mid+1, high)  
    else if (A[mid] > x) return binarySearch(A, x, low, mid-1)  
    else return mid
```

최선 Case $O(1)$

최악 ?

$\cdot n \quad n/2 \quad n/4 \quad n/8$

$O(n^2)$ 점근적 상한 : 기껏해야 n^2
 $0.000001n^2, 1000n, n \log n$

$\Omega(n^2)$ 점근적 하한 : 적어도 n^2
 $0.001n^3, n^4, 2^n$

$\Theta(n^2)$ 점근적 동일 : 항상 n^2 비례
 $0.001n^2, 1000n^2$

3가지 개념 중요

1. $n^2 = O(n^2)$ (○)

2. $n = \Omega(n^2)$ (×)

3. $n^2 = O(n^3)$ (○)

4. $2n^2 - 100n = \Omega(n^2)$ (○)

5. $3n^2 + \log n = O(n^2)$ (○)

6. $n^2 = O(n^{100})$ (○)

7. $7 \log n = O(n^{100})$ (○)

8. $5n^2 + 7n - 3 = \Theta(n^2)$ (○)

9. $5n^2 + 7n - 3 = \Omega(n \log n)$ (○)

10. $7n - 3 = \Theta(n^2)$ (×)